

INTERNSHIP

Cybersecurity project Assignment

- Intern ID : NGS-CS26-KM23
 - Project Name : **Keylogger** (Keystroke Monitoring Tool)
 - Assigned Mentor Name : **Mouresh**
-

Project:

[Keylogger_KM23.zip](#)

```
|
|— Keylogger/
|   |— keylogger.py # command-line version
|   |— gui.py # desktop GUI application
|   |— decrypt_log.py
|   |— requirements.txt
|   |— README.md
|   |— REPORT.md
|   |— LICENSE
|   |— .gitignore
|   |— docs/
|   |— logs/
```

Repository:

https://github.com/kartik-reddy-m/key_logger-encoding-decoding.git

Installation:

System Requirements

- Python 3.12 or later
- pip (Python Package Manager)
- Internet connection (for installing dependencies)
- Windows 10/11, macOS 12+, or Linux

Get the Project

Clone the repository from GitHub:

```
git clone https://github.com/kartik-reddy-m/key_logger-encoding-decoding.git
```

...or download and extract **Keylogger_KM23.zip** (attached to this PDF). Then open a terminal and enter the project folder:

```
cd Keylogger
```

Windows Installation

Step 1: Install Python

Download and install Python from:

- <https://www.python.org/downloads/>

During installation:

- ☒ Check "Add Python to PATH"
- Click **Install Now**

Step 2: Verify Python Installation

Check Python:

```
python --version  
or  
py --version
```

Check pip:

```
pip --version
```

Step 3: (Recommended) Create a Virtual Environment

```
python -m venv venv  
venv\Scripts\activate
```

If PowerShell blocks activation, run:

```
Set-ExecutionPolicy -Scope Process RemoteSigned
```

Step 4: Install Required Packages

```
pip install -r requirements.txt
```

You should see packages such as: **pynput**, **cryptography**.

Step 5: Run the Application

```
python keylogger.py  
or  
py keylogger.py
```

macOS Installation

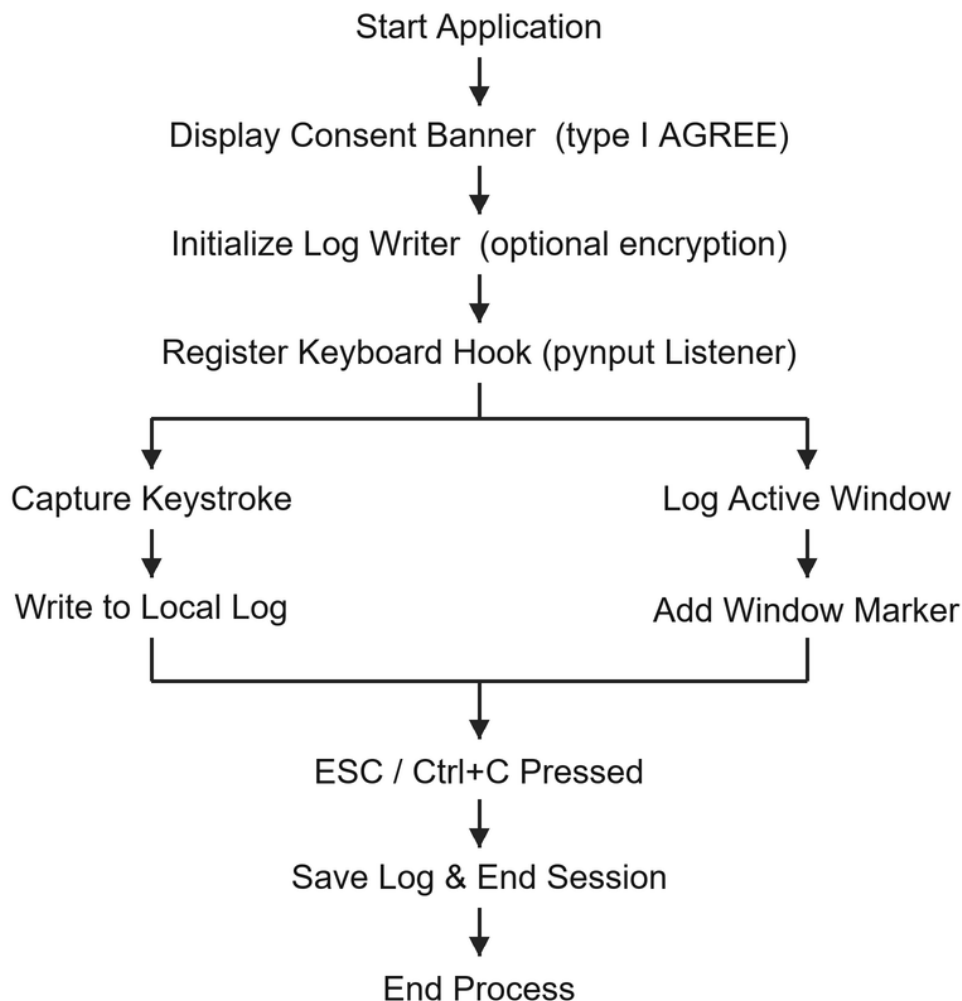
```
python3 --version          # or: brew install python  
python3 -m venv venv  
source venv/bin/activate  
pip install -r requirements.txt  
python3 keylogger.py
```

On macOS, grant your terminal **Accessibility** and **Input Monitoring** permission in System Settings > Privacy & Security.

Linux Installation (Ubuntu/Debian)

```
sudo apt update  
sudo apt install python3 python3-pip python3-venv xdotool -y  
python3 -m venv venv  
source venv/bin/activate  
pip install -r requirements.txt  
python3 keylogger.py
```

Project Workflow :

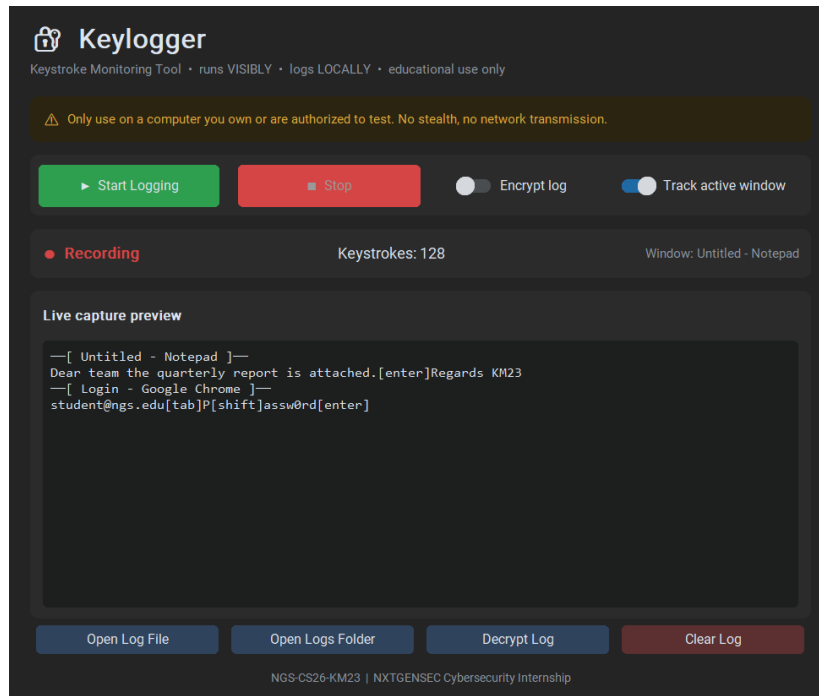


Description:

Keylogger (Keystroke Monitoring Tool) is a Python-based application developed to demonstrate how keystroke capture works using an operating-system keyboard hook (via the pynput library). It records key presses to a local, timestamped log and tags each entry with the active window title for context. The tool is built responsibly for educational use: it runs visibly, requires the operator's consent, stores data only on the local machine with optional Fernet encryption, and performs no network transmission. It is provided both as a command-line tool and as a desktop GUI application (customtkinter). This project demonstrates the practical application of input monitoring, cryptography, and cybersecurity concepts in understanding, detecting, and defending against keyloggers.

Screenshots:

GUI Application:



The desktop application: Start/Stop controls, live keystroke counter, capture preview, active-window tracking, and encrypt/decrypt tools.

Command-Line — Application Running:

```
Windows PowerShell

PS C:\Users\karth\...\keylogger-edu-project> python keylogger.py

=====
EDUCATIONAL KEYLOGGER - runs VISIBLY, logs LOCALLY
=====

This program will record the keys pressed on THIS computer
to a local file until you press ESC or Ctrl+C.

* It does NOT hide itself.
* It does NOT transmit anything over the network.
* Only use it on a machine you own or are authorized to test.
=====

Type "I AGREE" to start logging on this machine: I AGREE

[running] Logging keystrokes. Press ESC or Ctrl+C to stop.
[running] keystrokes captured: 128
```

The CLI version runs visibly and requires consent before logging.

Captured Log File:

```
Windows PowerShell

PS C:\...\keylogger-edu-project> type logs\keystrokes.log

===== SESSION START 2026-07-15 18:42:07 =====

[2026-07-15 18:42:09] --- WINDOW: Untitled - Notepad ---
Dear team
the quarterly report is attached.
Regards[enter]KM23

[2026-07-15 18:42:41] --- WINDOW: Login - Google Chrome ---
student@ngs.edu[tab]P[shift]assw0rd[enter]

===== SESSION END 2026-07-15 18:43:12 =====
```

Keystrokes saved locally with timestamps and active-window markers.

Encrypted Log & Decryption:

```
Windows PowerShell

PS C:\...\keylogger-edu-project> python keylogger.py --encrypt
[crypto] generated new key: logs\secret.key
[done] Log is encrypted. Decrypt it with: python decrypt_log.py

PS C:\...\keylogger-edu-project> type logs\keystrokes.log
gAAAAABm9x2kQ3pV7nT0oQ2lYc8bJ1rZ5wD... (unreadable ciphertext)
gAAAAABm9x2kL8sPq0aZ4mN6tR9uH2vX1cB...

PS C:\...\keylogger-edu-project> python decrypt_log.py
--- Decrypted contents of logs\keystrokes.log ---
===== SESSION START 2026-07-15 18:50:03 =====
[2026-07-15 18:50:05] --- WINDOW: Untitled - Notepad ---
hello world this is recovered plaintext
--- end ---
```

With --encrypt the log is stored as ciphertext and recovered via decrypt_log.py.

Features:

- **Consent-gated & visible** - refuses to run until the operator types *I AGREE*, and prints a live keystroke counter so operation is obvious.
- **Local-only** - writes solely to `logs/keystrokes.log`; performs no network transmission.
- **Active-window context** - tags each entry with the foreground application title, so the log shows *which* app received the typing.
- **Special-key handling** - printable keys are logged as characters; keys like Enter, Tab and Shift are written as readable tokens (e.g. `[shift]`).
- **Log rotation** - rotates the log at ~1 MB to keep files manageable.
- **Optional encryption** - `--encrypt` stores each line as Fernet ciphertext; `decrypt_log.py` recovers it.
- **Cross-platform** - runs on Windows, macOS and Linux.
- **Ethical guardrails** - no persistence, no stealth, no exfiltration by design.

Detection & Defense:

Because this project is defensive in purpose, it also documents how such a tool is spotted and stopped on a real system.

How a keylogger is detected

- **Global keyboard hook** - EDR flags a process installing `SetWindowsHookEx` (Windows) or requesting *Input Monitoring / Accessibility* (macOS).
- **Unexpected outbound traffic** - small periodic POSTs or odd DNS queries expose the *exfiltration* half of real spyware, even when capture is silent.
- **Suspicious file writes** - a process writing steadily to an obscure file in `%APPDATA%`, `/tmp`, or a hidden folder.
- **Persistence artifacts** - new Run registry keys, scheduled tasks, services, launch agents, or startup-folder entries.
- **Host review** - AV/EDR signatures and behavioral ML; Autoruns / Task Manager / `tasklist` inspection; and physical checks for inline USB loggers.

How to defend against it

- **Multi-factor authentication** - a stolen password alone becomes far less useful.
- **Password managers** - autofill credentials so secrets are never typed.
- **Endpoint hardening** - updated AV/EDR, application allow-listing, and least-privilege accounts so malware cannot install drivers or services.

- **OS hygiene** - patch promptly, review autoruns and drivers, enable Windows tamper protection / macOS SIP.
- **Network controls** - egress filtering and monitoring to catch exfiltration even if capture is missed.
- **High-risk sessions** - on-screen keyboards or a clean, dedicated device for sensitive logins; inspect shared machines and lock down USB ports.

Reference: MITRE ATT&CK T1056.001 - Input Capture: Keylogging. Full analysis in REPORT.md.

Thank You